THE UNIVERSITY OF MELBOURNE

COMP30022

Handover

Remaining Assessment

- Progress Report 3, due end of Week 12 (same format as Progress Reports 1 and 2)
- Oral Presentation, held next week
  - Please inform your clients
- Final product report, due November 7
- Individual contribution report, due November 7
- Ethics and security report, due November 12

Other Matters

- Week 12 Lecture: Subject Retrospective
- ESS Survey Now Available

the Handover
This is my code
It's your Problem now
I'm out.

Scheduling Handover

- Arrange a convenient meeting time between weeks 12 and 14, allowing time for discussion and demonstration
- Plan a structured presentation that covers all critical aspects of the application, including implemented user stories and any features or user stories that were not completed.
- Include a live demonstration of the application to showcase its functionality and how it meets the project requirements.

Handover

- How do you do a project handover?
- Handover is a crucial process in the software lifecycle, and it may turn out to be costly and harsh unless you take the appropriate precautionary steps to make it smooth and downtime-free
- It is not something you need to think about in your university assignments.
- Without documentation, even the best software developers are forced to analyze the entire source code on their own, which is a time-consuming and sometimes frustrating task.
- You will not receive a final product mark unless you have conducted handover

Conducting the handover meeting

- **Demonstrate the Application:** Walk the client through the application, emphasising how it addresses the initial project requirements and any specific functionalities that were a focus of the development.
- **Discuss the Code and Documentation:** Guide the client through the code repository, highlighting the structure, key components, and any important documentation that accompanies the code.
- **Review Outstanding Items:** Clearly communicate any unimplemented features or user stories, including reasons why they were not completed and suggestions for future development.
- **Discuss Accounts, API keys, etc…**: If your software uses third party accounts or API keys, make sure the client has control of these
- **Arrange Deployment on client infrastructure (if required)**
- **Feedback and Questions:** Allocate time for the client to provide feedback and ask questions. This is crucial for ensuring they fully understand the application and what has been delivered.

Handover (checklist to be used as a reference)

Codebase: Include all source code developed during the semester, ensuring it's the latest and most stable version.

License: Ensure the codebase is accompanied by an appropriate license agreement, transferring ownership rights to the client as agreed upon. Also describe any licenses for third party software your code relies on.

README Files: Populate README files with detailed descriptions of the repository structure, making it easier for the client or future developers to navigate the project.

Database Access: Provide comprehensive details on accessing the database, including necessary credentials and an overview of the data schema.

Hosting Service Access: Include instructions and credentials for accessing the hosting service where the application is deployed.

Description of the database structure (additionally, in case of a relational database, a description of their correlations should also be included; this can be presented on ERP - Entity Relationship Diagram),

Data Documentation: Document all data stored within the database, particularly emphasising test data or dummy user information.

Handover (checklist to be used as a reference)

Description of key classes and the application's layers (this simply makes it easier for the new team to find the place that requires changes),

Description of key algorithms (e.g. calculating the service price or any algorithms that are unusual) + mechanism of importing data from 3rd party systems,

Project Documentation (e.g. user stories, personas, motivational model)**:** Export and organise the entire document repository, ensuring all project documentation is easily accessible, including design documents, meeting minutes, and development logs.

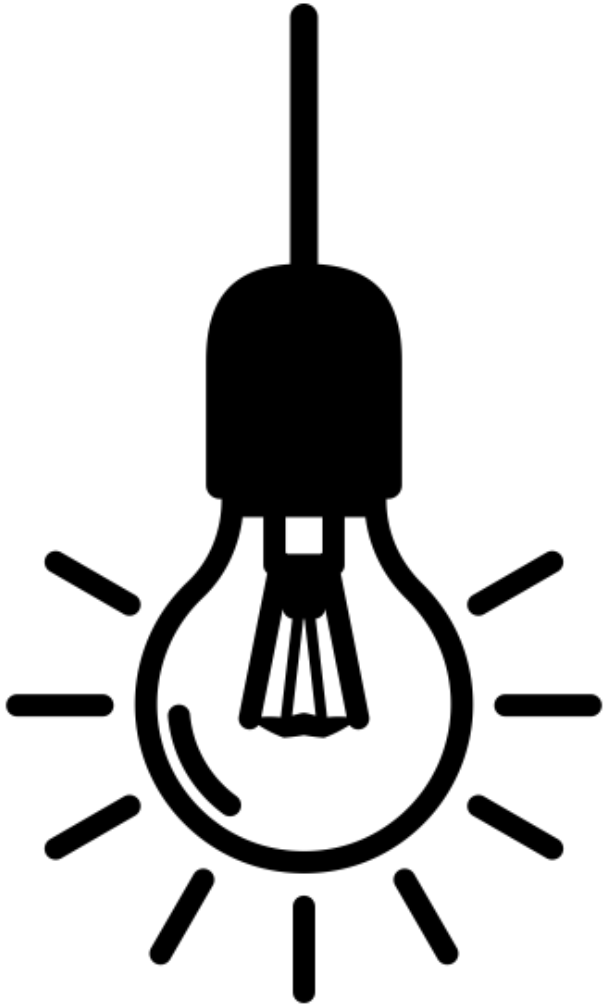
User Manual: Create a user manual highlighting the system's use cases, ideally leveraging your acceptance testing documentation to guide the client through the application's features and functionalities.

Startup and Shutdown Instructions: Provide clear instructions for starting up and shutting down the application, ensuring the client can easily manage the application's operational status.

Deployment guidelines (i.e. how to deploy the source code, database and run the project on a new server, <u>any necessary administrator/test customer login credentials</u>, access to code repositories, databases and servers as well

After the handover meeting

- **Provide Contact Information:** While the formal project period may be over, it’s helpful to offer contact information for further queries or clarifications.
- **Follow-Up:** Send a follow-up email thanking the client for their collaboration, including a summary of the handover meeting and links to the repository and documentation for easy reference.
- More than a formality; the final step in delivering a product that could see real-world use. By carefully preparing and conducting the handover meeting, you ensure that your project has a lasting impact beyond the academic environment.



Confluence

(or similar collaboration tool)

Create a ‘Handover’ page and link it to your code repository

Handover – Table of contents example

Table of Contents

TL;DR

Installation Options

- 1 - Manual
- 2 - Release Archive Download
- 3 - Install Script
- 4 - Homebrew Fonts (macOS (OS X))
- 5 - Clone Repo
- 6 - Ad Hoc Curl Download
- 7 - Arch User Repository (AUR) (Arch Linux)
- 8 - Patch Your Own Font

Features

- Glyph/Icon sets
- Patched Fonts
- Combinations
- Font Patcher

Developer / Contributor

- Font Patcher
- Gotta Patch 'em All Font Patcher!
- Other Good Fonts to Patch
- Contributing

Project Motivation

Additional Info

- Unstable file paths on master
- Changelog
- License

Example of table of contents

[example on github]

Table of contents

Project background/overview
Demo (link to hosted project)
Features (user stories – organized in sprints)
Documentation (user stories, architecture, test cases, other docs from Confluence – exported as PDF)
System requirements (tools/database and their versions)
Installation guide (setup and configuration details to install/run your code)
Changelog
Traceability matrix

[example for this subject]

Example of readme

Handover – Changelog example

Release History

- 1.1.3 - Fix for Node 10 support (@aredridel). Officially dropping support for Node < 4.
- 1.1.2 - Prevent more `ENOENT` errors from escaping (@alexgorbatchev).
- 1.1.1 - Prevent `fs.watch` errors from escaping error handler (@rosen-vladimirov). Fix `_addToWatched` without `path.sep` (@wyicwx).
- 1.1.0 - Update to `globule@1.0.0` with `minimatch >= 3.0.0`.
- 1.0.0 - Revert back to 0.5.2. Drop support for Node.js v0.8. Fix for `maxListeners`. Update `globule` to `0.2.0`.
- 0.6.4 - Catch and emit `error` from `readdir` (@oconnore). Fix for `0 maxListeners`. Use `graceful-fs` to avoid `EMFILE` errors in other places `fs` is used. Better method to determine if `pathwatcher` was built. Fix keeping process alive too much, only init `pathwatcher` if a file is being watched. Set min required to Windows Vista when building on Windows (@pvolok).
- 0.6.3 - Add support for Node.js v0.11
- 0.6.2 - Fix argument error with `watched()`. Fix for erroneous `added` events on folders. Ignore `msvs` build error 4244.
- 0.6.1 - Fix for absolute paths.
- 0.6.0 - Uses native OS events (fork of `pathwatcher`) but can fall back to stat polling. Everything is async to avoid blocking, including `relative()` and `watched()`. Better error handling. Update to `globule@0.2.0`. No longer watches `cwd` by default. Added `mode` option. Better `EMFILE` message. Avoids `ENOENT` errors with symlinks. All constructor arguments are optional.
- 0.5.2 - Fix for `ENOENT` error with non-existent symlinks [BACKPORTED].
- 0.5.1 - Use `setImmediate` (`process.nextTick` for Node.js v0.8) to defer `ready` / `nomatch` events (@amasad).
- 0.5.0 - Process is now kept alive while watching files. Emits a `nomatch` event when no files are matching.
- 0.4.3 - Track file additions in newly created folders (@brett-shwom).
- 0.4.2 - Fix `.remove()` method to remove a single file in a directory (@kaelzhang). Fixing "`Cannot call method 'call' of undefined`" (@krasimir). Track new file additions within folders (@brett-shwom).

[example on github]

Example of changelog

It should be consistent to the listed features

Changelog (or release history)

Sprint 2
feature development from user story[branch]
bugfix at user story
feature development from user story[branch]
feature development from user story[branch]
feature development from user story[branch]
bugfix at user story

Sprint 1
feature development from user story[branch]
feature development from user story[branch]
feature development from user story[branch]
bugfix at user story

[example for this subject]

Handover – Release example

Releases | Tags

Edit release | Delete

Latest release
3.0.0
09559e5
Verified
Compare

Sprint 3 Release

eric-sciberras released this 11 days ago · 2 commits to master since this release

Summary

Application Deployed Here

The Third Sprint goal for the Front-end has been to allow for a round to be played as a means to show a slimmed down version of the final product.

We have added:

- Further Integration of the Backend API, allowing for a game to now be played
- Additional pages such as the *Discussion* and *Individual Background* pages
- The ability to rejoin the game, if the user accidentally or purposely leaves
- Additional unit tests and automated the running of these tests as part of our review process
- Minor UI Fixes

List of changes made in sprint 3

Format: [Jira Task ID] [User Story] [Short Description]

- CE90013-223 US_07 frontend: Role background page(with goal)
- CE90013-221 US_08 frontend_01: Chapter page_general information
- CE90013-222 US_08 frontend_02: Chapter page_individual information
- CE90013-224 US_10 frontend: Create the 'decision'/'discussion' page
- CE90013-226 US_12 frontend: See all decisions connect with backend
- CE90013-227 US_14 frontend: Refine error message box(other APIs)
- CE90013-225 US_15 frontend: Create the 'enter choice'/'result' page
- CE90013-206 US_20 frontend_01: Merge role description and choose your role pages
- CE90013-220 US_20 frontend_02: Fix bugs introduced from UI refactor
- CE90013-208 US_20 frontend_03: Session handling
- CE90013-228 US_21 frontend_01: Add test suites to test what we have achieved in sprint2 and some of sprint3 features
- CE90013-229 US_21 frontend_02: update testing plan and testing results for sprint3

Assets 2

Source code (zip)
Source code (tar.gz)

Handover – Release example

Releases | Tags

Tags

3.0.0 ...
11 days ago 09559e5 zip tar.gz Notes
Verified ...

2.0.0 ...
on Sep 1 1189e8e zip tar.gz Notes
Verified ...

v1.0.0 ...
on Jun 10 5ab364b zip tar.gz
Verified ...

Other matters

- Even if you're not using Confluence, most collaboration software should have similar functionality.
- If not, export required documents as PDFs
- Ensure the client can access and modify the codebase moving forward
- Transferring GitHub repository ownership is sensible